PRD: Encrypted Peer-to-Peer File & Text Transfer

Author: Madhav | **Status:** Draft v0.1 | **Date:** 9 Oct 2026

1. Overview

A web app that moves files and text directly between two devices, encrypted end to end, with no server ever seeing or storing the content. Devices pair by scanning a QR code or typing a short code. The only server involved is a tiny signaling service that helps two browsers find each other; it never touches the data.

One-line pitch: Open the site on two devices, scan or type a code, drop a file. It goes straight from one to the other, and nobody in the middle can read it.

2. Problem Statement

Sending a file from a phone to a laptop (or to a friend's device) today means one of: emailing it to yourself, uploading to WhatsApp/Drive/WeTransfer, or using a cable. All of these route data through a third party's servers, add friction, compress or limit files, and leave copies behind. Existing P2P tools (AirDrop, Nearby Share) are locked to ecosystems; browser tools like Snapdrop depend on being on the same network and offer weak pairing security.

3. Goals and Non-Goals

Goals

- Transfer files and text between any two devices with a modern browser, with no install and no account.
- End-to-end encryption by design: the server cannot read content even if it is malicious or compromised.
- Pairing in under 15 seconds via QR or short code.
- Works across different networks (not only the same Wi-Fi).
- Clean enough codebase and write-up to be a strong GitHub and resume project.

Non-Goals (v1)

- No user accounts, login, or cloud storage of any kind.

- No group or multi-peer rooms (1-to-1 only).
- No offline delivery (both devices must be online at the same time).
- No native mobile or desktop apps (PWA only).
- No anonymity against network observers (IP addresses are visible to the peer and to STUN/TURN).

4. Target Users

Persona	Need
Student (primary)	Move notes, PDFs, code and screenshots between phone, laptop, and college lab PCs without logging into personal accounts.
Privacy-conscious user	Share a sensitive document without it landing on a company server.
Cross-ecosystem pair	Android to Windows, iPhone to Linux, where no built-in tool works.

5. Key User Flows

5.1 Pair by QR code

1. Device A opens the site and taps **Create session**. A QR code and a short code are displayed.
2. Device B scans the QR with its camera, which opens the site with the session secret in the URL fragment.
3. Both devices show a short verification phrase to compare (optional but encouraged).
4. Connection status turns to **Connected**.

5.2 Pair by short code

1. Device A shows a code such as `7-kiwi-lamp`.
2. Device B opens the site, chooses **Join**, and types the code.
3. Pairing completes as above.

5.3 Send a file or text

1. Either side drags a file onto the page (or picks one, or pastes text).
2. Receiver sees filename, size, and an **Accept** / **Decline** prompt.
3. Transfer begins with a progress bar, speed, and ETA.
4. Receiver gets a download; both sides can send more until the session is closed.

5.4 End session

Either side taps **Disconnect**, or the session expires. All keys are discarded from memory.

6. Functional Requirements

P0: Must have (MVP)

ID	Requirement
F1	Create a session and display a QR code plus a human-readable short code.
F2	Join a session by scanning the QR or entering the code.
F3	Establish a direct browser-to-browser connection using WebRTC data channels.
F4	Send and receive files of at least 2 GB using chunked streaming (no loading the whole file into memory).
F5	Send and receive plain text snippets.
F6	Receiver must explicitly accept an incoming file before any bytes are saved.
F7	Progress, speed, and cancel controls for each transfer.
F8	Sessions are single-use and expire after 5 minutes if unpaired.
F9	Clear connection states: waiting, connecting, connected, failed, closed.

P1: Should have

ID	Requirement
F10	Multiple files in one transfer, queued.
F11	Verification phrase (short authentication string) shown on both devices to detect a man-in-the-middle.
F12	Integrity check: SHA-256 of each file verified on the receiver.
F13	Installable PWA with share-target so files can be sent from the phone's share sheet.
F14	Resume an interrupted transfer within the same session.
F15	TURN relay fallback when direct connection is impossible (still end-to-end encrypted).

P2: Nice to have

- Folder transfer as a zip stream.
- Optional passphrase on top of the code for high-security sessions.
- Dark mode, localization.
- Clipboard sync mode.

7. Security and Privacy Requirements

This is the core of the product, so it gets its own section.

1. **No plaintext on the server, ever.** The signaling server only relays connection-setup messages (SDP offers/answers, ICE candidates). It never receives file bytes or filenames.
2. **Authenticated key exchange from the pairing secret.** The QR code carries a high-entropy secret in the URL fragment (`#...`), which browsers never send to servers. For the short typed code, which has low entropy, use a password-authenticated key exchange (PAKE) such as SPAKE2 or CPace, so that an attacker cannot brute-force the code offline from captured messages.

3. **Layered encryption.** WebRTC already encrypts data channels with DTLS. On top of that, add application-level encryption (AES-256-GCM via the Web Crypto API) with keys derived from the PAKE output, and bind the DTLS fingerprints into the key derivation so a malicious signaling server cannot swap in its own endpoint.
4. **Short-lived and single-use sessions.** Codes expire in 5 minutes, work once, and are rate-limited per IP on the signaling server to block guessing.
5. **Ephemeral keys.** Fresh keys per session; nothing persisted to disk or localStorage. This gives forward secrecy at the session level.
6. **No tracking.** No analytics that capture filenames, sizes, or IPs. Server logs are minimal and rotated.
7. **Transparent threat model.** Publish what the app protects against and what it does not (see Section 13).

8. Recommended Technical Approach

My recommendation for v1, with reasons:

Layer	Choice	Why
Transport	WebRTC `RTCDataChannel`	Direct browser-to-browser, NAT traversal built in, no relay of data in the common case.
Signaling	Small Node.js server using WebSockets (single process, in-memory sessions)	Easy to build and host free; stateless apart from short-lived session entries.
NAT traversal	Public STUN to start; self-hosted coturn as the TURN fallback in P1	STUN covers most home and mobile networks; TURN only needed for strict NATs and college firewalls.
Pairing crypto	CPace or SPAKE2 (WASM/JS library) for the short code; random 128-bit secret for QR	Prevents offline guessing of the short code. Do not invent your own protocol.
Payload crypto	Web Crypto AES-GCM with per-chunk nonces	Native, fast, audited primitives.
Frontend	React or plain TypeScript plus Vite; PWA manifest and service worker	Fast build, installable, works on phone browsers.
File handling	Streams API and File System Access API where supported; Blob fallback otherwise	Needed for multi-GB files without crashing the tab.
Hosting	Static frontend on any static host; signaling on a small free-tier server	Near-zero cost.

Why WebRTC over a custom TCP/WebSocket relay: a relay means the server carries the bytes, which defeats the "no middleman" promise and costs bandwidth. WebRTC keeps the data path peer to peer.

Chunking: 16 KB chunks (the safe cross-browser data channel message size), with backpressure using `bufferedAmountLowThreshold` so the sender never overruns the channel.

9. High-Level Architecture

- **Client A and Client B:** browsers running the same web app.
- **Signaling server:** creates a session ID, relays handshake messages between exactly two clients, then forgets the session.
- **STUN/TURN:** helps the clients discover routes to each other; TURN relays only already-encrypted bytes if needed.
- **Data path:** after the handshake, all content flows over the encrypted data channel directly between A and B.

10. Non-Functional Requirements

Area	Target
Pairing time	Under 15 seconds from opening the site to connected, on a normal connection.
Throughput	Within 70% of the available link speed on a LAN; usable (above 2 MB/s) across the internet when direct.
Memory	Constant memory use regardless of file size (streaming).
Browser support	Latest Chrome, Edge, Firefox, Safari (iOS 16+), and Chrome on Android.
Availability	Signaling server 99% (best effort for a student project).
Accessibility	Keyboard operable, readable contrast, QR alternative via typed code.
Mobile	Fully usable at 360 px width; transfers continue while the screen is on.

11. Success Metrics

Since there is no tracking, metrics are measured in testing and through opt-in feedback rather than analytics.

- Pairing success rate in test runs across at least 10 device and network combinations: above 95%.
- Median time to first byte after scan: under 10 seconds.
- A 1 GB file transfers successfully and the hash matches in 100% of test runs.
- Zero plaintext content or filenames visible in server logs (verified by inspection).
- Project outcomes: public GitHub repository with a README, architecture diagram, and threat model; live demo link.

12. Milestones

Phase	Scope	Rough timing
0. Research	Learn WebRTC data channels, ICE/STUN/TURN, Web Crypto; skim the CPace/SPAKE2 idea.	Week 1
1. Proof of concept	Two tabs connect through a signaling server and exchange text.	Week 2
2. File transfer	Chunking, backpressure, progress, accept/decline, multi-GB streaming.	Weeks 3-4
3. Pairing and crypto	QR and short code flows, PAKE, app-layer encryption, verification phrase.	Weeks 5-6
4. Polish and PWA	UI, error states, PWA install, share target, TURN fallback.	Weeks 7-8
5. Launch	Threat model write-up, README, deployment, demo video, resume entry.	Week 9

13. Threat Model Summary

Threat	Mitigation
Malicious or compromised signaling server	Keys derived from the pairing secret; DTLS fingerprints bound into the key; server cannot decrypt or silently substitute an endpoint.
Attacker guessing the short code	PAKE blocks offline guessing; single-use codes, 5-minute expiry, and rate limits block online guessing.
Shoulder-surfing the QR or code	Single use plus the verification phrase; the first successful pairing consumes the session.
Network eavesdropper	DTLS plus application-layer AES-GCM.
Malicious file from the peer	Receiver must accept; no auto-open; filename sanitized; size and type shown before accepting.
Peer IP exposure	Documented limitation; optional TURN-only mode in P2.
Denial of service on signaling	Per-IP rate limits and small session lifetime.

14. Risks and Open Questions

- **Strict college or corporate networks** may block UDP and WebRTC. Mitigation: TURN over TCP/443 (P1). Decision needed on who pays for a TURN server.
- **iOS Safari limits** on background tabs and large-file downloads may constrain multi-GB transfers on iPhone. Needs early testing.
- **PAKE library maturity in JavaScript.** Choose an audited implementation; if none is acceptable, fall back to a long random code (about 10+ characters) and drop PAKE for v1.
- **Resume support** adds protocol complexity; decide in Phase 2 whether it stays in P1.
- **Naming and branding** are undecided.

15. Out of Scope for Now (Future Ideas)

- Native Android and desktop clients.

- Persistent trusted devices (pair once, reconnect later).
- Group rooms and one-to-many broadcast.
- Encrypted store-and-forward when the receiver is offline.

16. Resume Framing

One-liner for the resume once built: *Built an end-to-end encrypted, serverless-data peer-to-peer file transfer web app using WebRTC, Web Crypto (AES-GCM), and PAKE-based QR/code pairing; streams multi-GB files with constant memory.*